

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

# 1. الكولاب - 10 سنين بيانات شهرية
np.random.seed(42)
dates = pd.date_range(start='2014-01-01', end='2024-01-01', freq='MS')

df = pd.DataFrame({
    'Date': dates,
    'Oil_Price_USD': np.random.uniform(50, 100, len(dates)),
    'Rigs_Count': np.random.randint(10, 100, len(dates)),
    'Temperature_C': np.random.uniform(-10, 40, len(dates)),
    'Maintenance_Days': np.random.randint(0, 30, len(dates))
})

# نسوي علاقة واقعية للإنتاج
df['Oil_Production_BBL'] = (
    50000 +
    df['Oil_Price_USD'] * 300 +
    df['Rigs_Count'] * 80 -
    df['Maintenance_Days'] * 2000 +
    np.random.normal(0, 5000, len(dates))
).astype(int)

print("تم توليد الداتا ✅")
print("شكل الداتا:", df.shape)
print("\nأسماء الأعمدة:", df.columns)
print("\nأول 5 صفوف:")
print(df.head())

# 2. تحضير الداتا للموديل
X = df[['Oil_Price_USD', 'Rigs_Count', 'Temperature_C']]
y = df['Oil_Production_BBL']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. نجرب موديلين ونقارن
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor()
}

```

```
for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    r2 = r2_score(y_test, preds)
    mae = mean_absolute_error(y_test, preds)
```

/tmp/ipykernel_467/730634818.py:11: FutureWarning: 'M' is deprecated and will be removed in a future version. Use 'date_range' instead.

dates = pd.date_range(start='2014-01-01', end='2023-12-01', freq='M')
تم توليد الداتا ✓
(6 ,119) شكل الداتا :

الأعمدة : أسماء : ['Date', 'Oil_Price_USD', 'Rigs_Count', 'Temperature_C', 'Maintenance_Count']

: أول 5 صفوف

	Date	Oil_Price_USD	Rigs_Count	Temperature_C	Maintenance_Count
0	2014-01-31	69.963210	427	41.761397	103599
1	2014-02-28	116.057145	530	33.934159	121630
2	2014-03-31	98.559515	489	38.844339	115395
3	2014-04-30	87.892679	524	30.079113	122205
4	2014-05-31	52.481491	582	32.307117	105777

	Oil_Production_BBL
0	103599
1	121630
2	115395
3	122205
4	105777

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# نخلي الشكل احترافي
sns.set_style("darkgrid")
plt.rcParams['figure.figsize'] = (12, 5)
```

```
# 1. رسم خط زمني للإنتاج
plt.figure(figsize=(14,6))
plt.plot(df['Date'], df['Oil_Production_BBL'], color='#1f77b4')
plt.title('Oil Production Over Time - BBL', fontsize=14, fontweight='bold')
plt.xlabel('Date')
plt.ylabel('Oil Production BBL')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# 2. علاقة سعر النفط بالإنتاج
plt.figure(figsize=(7,5))
sns.scatterplot(x='Oil_Price_USD', y='Oil_Production_BBL', data=df)
plt.title('Oil Price vs Production', fontsize=14, fontweight='bold')
```

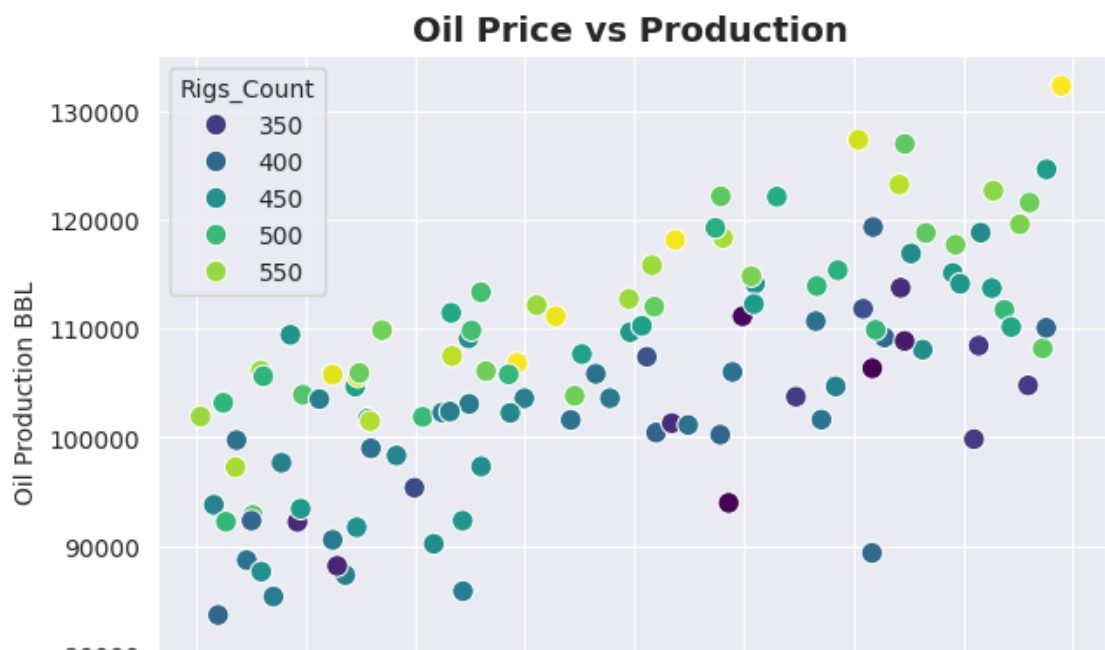
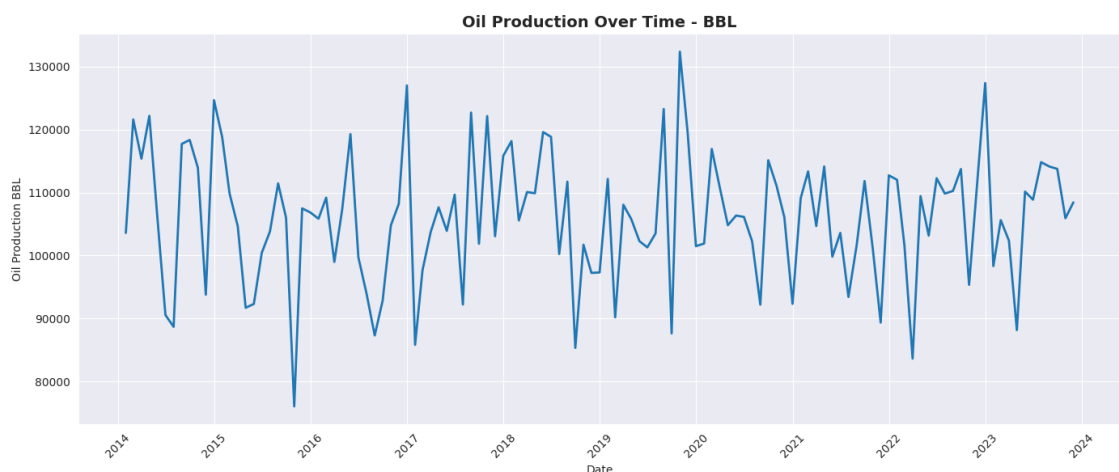
```
plt.xlabel('Oil Price USD')
plt.ylabel('Oil Production BBL')
plt.show()
```

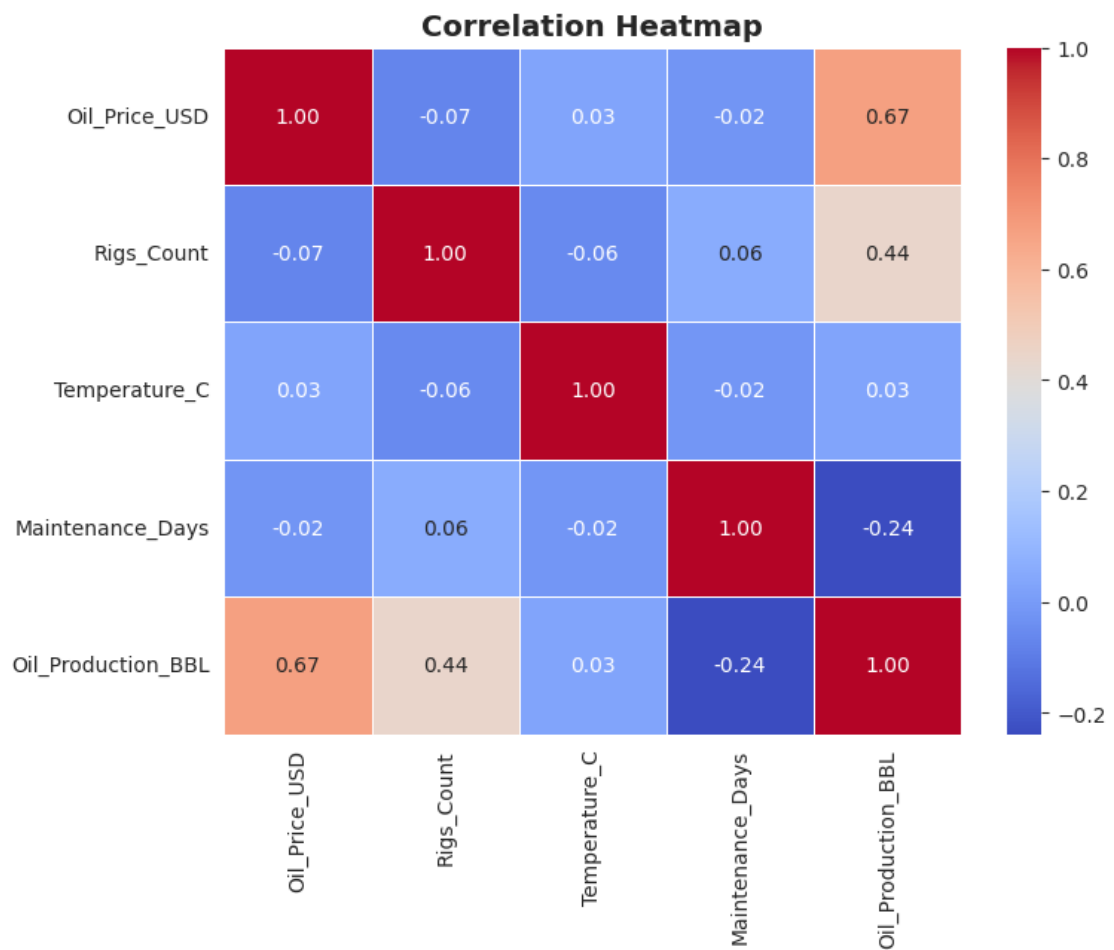
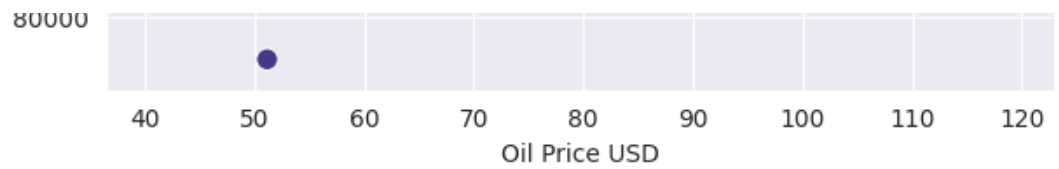
3. Heatmap للارتباط بين الأعمدة

```
plt.figure(figsize=(8,6))
corr = df.select_dtypes(include=np.number).corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f", line
plt.title('Correlation Heatmap', fontsize=14, fontweight='bold')
plt.show()
```

4. تأثير أيام الصيانة على الإنتاج

```
plt.figure(figsize=(7,5))
sns.boxplot(x='Maintenance_Days', y='Oil_Production_BBL', data=
plt.title('Maintenance Days Impact on Production', fontsize=14
plt.xlabel('Maintenance Days')
plt.ylabel('Oil Production BBL')
plt.show()
```





/tmp/ipykernel_467/1456204514.py:35: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will

`sns.boxplot(x='Maintenance_Days', y='Oil_Production_BBL', data=`

Maintenance Days Impact on Production





```
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# 1. من جديد عشان نتأكد انه موجود Random Forest ندرّب
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# 2. نجيب أهمية كل ميزة
importances = rf_model.feature_importances_
features = X.columns

feat_imp = pd.DataFrame({
    'Feature': features,
    'Importance': importances
}).sort_values('Importance', ascending=False)
```

```
print("أهمية المتغيرات:")
print(feat_imp)
```

3. نرسمها

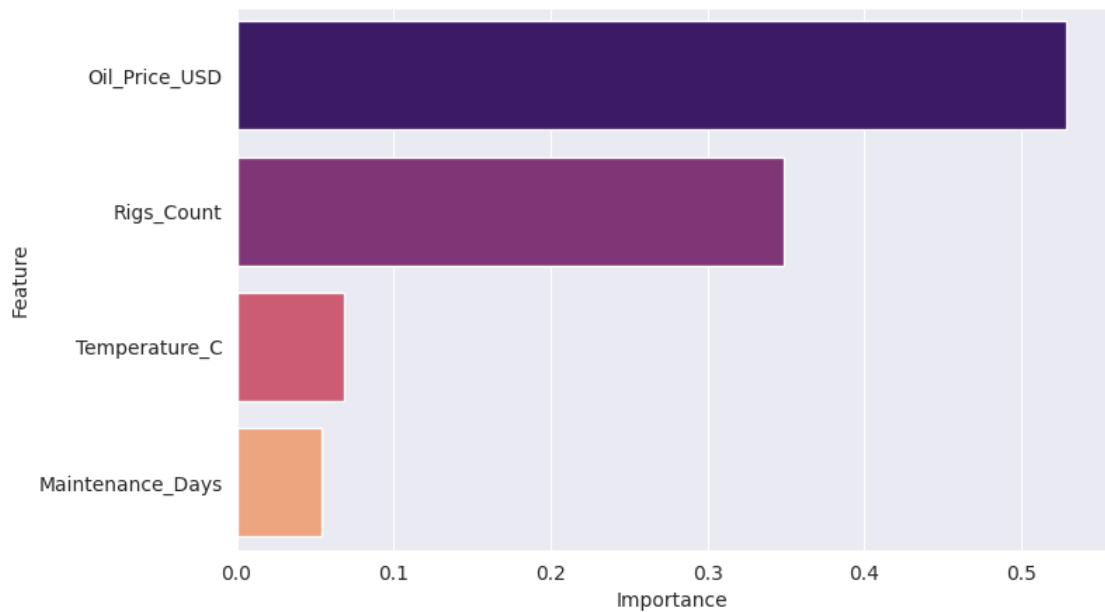
```
plt.figure(figsize=(8,5))
```

```
sns.barplot(x='Importance', y='Feature', data=feat_imp, hue='f
```

أهمية المتغيرات:

	Feature	Importance
0	Oil_Price_USD	0.528700
1	Rigs_Count	0.349012
2	Temperature_C	0.068341
3	Maintenance_Days	0.053946

```
<Axes: xlabel='Importance', ylabel='Feature'>
```



```
# لحفظ كل الصور مرة وحدة Cell
import matplotlib.pyplot as plt
import seaborn as sns
```

0. لازم نعرف المصفوفة بالأول

```
corr_matrix = df.corr() # لسطر <--
```

1. Feature Importance

```
plt.figure(figsize=(8,5))
```

```

plt.figure(figsize=(8,5))
sns.barplot(x='Importance', y='Feat
plt.title('Feature Importance for O
plt.tight_layout()
plt.savefig('feature_importance.png
plt.close()

# 2. Correlation Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.savefig('correlation_heatmap.pr
plt.close()

# 3. Price vs Production
plt.figure(figsize=(8,5))
sns.scatterplot(x='Oil_Price_USD',
plt.title('Oil Price vs Production
plt.tight_layout()
plt.savefig('price_vs_production.pr
plt.close()

# 4. Rigs vs Production
plt.figure(figsize=(8,5))
sns.scatterplot(x='Rigs_Count', y='
plt.title('Rigs Count vs Production
plt.tight_layout()
plt.savefig('rigs_vs_production.png
plt.close()

# 5. Maintenance Boxplot
plt.figure(figsize=(8,5))
sns.boxplot(x='Maintenance_Days', y
plt.title('Maintenance Days vs Prod
plt.tight_layout()
plt.savefig('maintenance_boxplot.pr
plt.close()

print("جاه! للملفات على اليسار")

```



تم حفظ كل الصور بنجاح! روجي للملفات على اليسار

```

import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files

# 1. بالاول feat_imp نعيد تعريف
feat_imp = pd.Series(rf_model.feature_importances_, index=X.columns)
feat_imp = feat_imp.reset_index()
feat_imp.columns = ['Feature', 'Importance']

# 2. أهم صورة - Feature Importance نحفظ
plt.figure(figsize=(8,5))
sns.barplot(x='Importance', y='Feature', data=feat_imp, hue='Feature')
plt.title('Feature Importance for Oil Production', fontsize=14)
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=300, bbox_inches='tight')
plt.close()

# 3. Correlation Heatmap نحفظ
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.savefig('correlation_heatmap.png', dpi=300, bbox_inches='tight')
plt.close()

# 4. تحميل تلقائي
files.download('feature_importance.png')
files.download('correlation_heatmap.png')

print("feature_importance.png و correlation_heatmap.png تم! بينزلوا صورتين الحين")

```

feature_importance.png و correlation_heatmap.png تم! بينزلوا صورتين الحين

